# How to Learn the Student Attendance System

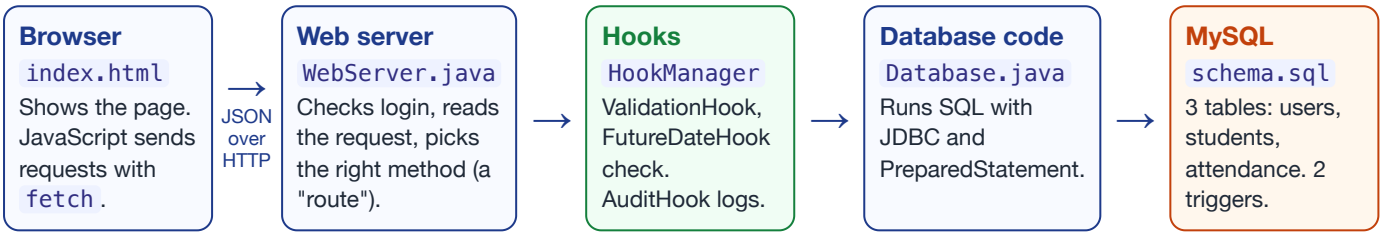
A short guide for beginners: what to read, in what order, and how to practise

## 1 What the app does

- A teacher **logs in**, then uses three tabs: **Students** (add, edit, delete), **Mark Attendance** (Present or Absent for the whole class, one Save) and **Attendance Report** (percentages, students below 75%, each student's day-by-day history, and an **Excel** download).
- Everything is saved in a **MySQL** database. The server is plain **Java** with no framework, and the page is one file of **HTML, CSS and JavaScript**.

## 2 The big picture

Every button follows the same path. Learn this picture first and every file will make sense.




The answer travels back the same way as JSON, and the page updates. `ExcelExporter.java` builds the .xlsx file, `StudentRecord.java` holds one student's numbers, and `Main.java` starts everything.

## 3 Before you start

**Basics that help**

- Java: classes, methods, `if`, `for`, `List`, exceptions
- SQL: `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `WHERE`
- HTML tags, a little CSS, JavaScript functions and variables

Don't know one of these? Spend an hour on a free tutorial first, then come back.

**Run it once** (from the project folder)

```
mysql -u root < schema.sql
mvn -q exec:java
mvn -q test
```

Open `http://localhost:8080` and log in as `admin` / `admin123`. Click every button once so you know what the code is for.

## 4 Read the files in this order

| # | File | What you learn there | Time |
|---|---|---|---|
| 1 | `schema.sql` | Tables, primary key, unique key, foreign key, `ENUM`, triggers, the sample data | 20 min |
| 2 | `db.properties`, `DbConfig.java` | How settings are kept outside the code and read at start | 10 min |
| 3 | `StudentRecord.java` | A simple data class, and the percentage formula | 10 min |
| 4 | `Database.java` | JDBC: connection, `PreparedStatement` with `?`, `ResultSet`, transactions, password check | 45 min |
| 5 | `Hook.java` → `HookManager.java` → the 3 hooks | Interfaces, default methods, how a "before" hook can stop an action | 30 min |
| 6 | `WebServer.java` | HTTP routes, status codes, building JSON, login sessions and cookies | 60 min |
| 7 | `index.html` | Page layout, CSS styling, JavaScript `fetch`, `async` / `await`, filling tables | 60 min |
| 8 | `ExcelExporter.java`, `Main.java` | Making an Excel file with Apache POI, how the program starts | 20 min |

**How to read a file:** read the comment at the top of the class, then the comment above each method, and only then the code. Every method's comment says which button uses it. Keep the app open and click that button while you read.

## 5 Follow one click from start to end

Pick the **Save Attendance** button and trace it through the code. Open each file at the named function.

1. `index.html` › `saveAttendance()` collects every checked button into text like `1:PRESENT,3:ABSENT`.
2. `postForm()` sends it with `fetch` to `POST /api/attendance/all`. The browser adds the login cookie by itself.
3. `WebServer.java` › `route()` checks the cookie (not logged in → 401) and calls `markWholeClass()`.
4. `markWholeClass()` checks every mark first, then runs `beforeMarkAttendance` on every hook. `FutureDateHook` can say no here, and then nothing is saved.
5. `Database.java` › `markAttendance()` runs an `INSERT` (new day) or `UPDATE` (day already marked).
6. `afterMarkAttendance` runs, and `AuditHook` writes a line to `hooks.log`.
7. The server answers with JSON like `{"ok":true,"saved":12,"present":11,"absent":1}` and the page shows the green message.

**Now do it yourself** for **Add Student** (`saveStudent()` → `addStudent()`) and for **Login** (`login()` → `checkLogin()`).

## 6 Words you will meet

| Word | Meaning in this project |
|---|---|
| HTTP request / response | The browser asks (GET to read, POST to change), the server answers with a status code and a body. |
| Status codes | 200 OK, 400 bad input, 401 not logged in, 403 refused, 404 not found, 409 duplicate roll number, 503 MySQL down. |
| Route | A URL the server knows, like `/api/students`. `route()` sends each one to its own method. |
| JSON | Text that carries data, like `{"name":"Asha","present":5}`. Java writes it, JavaScript reads it. |
| `fetch`, `async` / `await` | JavaScript's way to call the server and wait for the answer without freezing the page. |
| JDBC, PreparedStatement | How Java talks to MySQL. The `?` placeholders keep user text separate from SQL, which stops **SQL injection**. |
| Transaction | Several SQL steps saved together with `commit()`, or undone together with `rollback()`. |
| Hook | A small class that runs before or after an action. A "before" hook can cancel it by throwing an exception. |
| Trigger | SQL that MySQL runs by itself. Here it turns the division into capitals when a student is added or edited. |
| Hash + salt | The password is stored as SHA-256(salt + password), never as plain text. Login hashes the typed password and compares. |
| Session cookie | After login the server gives the browser a random token. The browser sends it back with every request. |

## 7 Practice: change something small

Doing is the fastest way to learn. Run `mvn -q test` after each change to check nothing broke.

| # | Task | Where to look |
|---|---|---|
| 1 | Make the minimum 80% instead of 75%. | `loadReport()` in `index.html` |
| 2 | Show "Total students: 12" above the report table. | `index.html` (HTML + `loadReport()`) |
| 3 | Write a new hook: the roll number must be digits only. | New class like `ValidationHook.java`, register it in `Main.java` |
| 4 | Add a second teacher account. | The `INSERT INTO users` example in `README.md` |

| | | |
|---|---|---|
| 5 | Add an Email column for students. | `schema.sql`, `Database.java`, `WebServer.java`, `index.html` |
| 6 | Harder: add a "Late" status. | The `ENUM` in `schema.sql`, the status check in `WebServer.java`, a third pill in `index.html` |

## 8 Questions you may be asked

**Why use PreparedStatement?** The values are sent separately from the SQL, so typed text can never change the query. This prevents SQL injection.

**How is the password stored?** As a salted SHA-256 hash in the `users` table. The real password is never saved.

**How does the server know you are logged in?** After login it makes a random token, remembers which user owns it, and puts it in an `HttpOnly` cookie. Every request must bring that cookie back.

**What is a hook?** A class that runs before or after an action. `ValidationHook` stops blank names, `FutureDateHook` stops future dates, `AuditHook` writes `hooks.log`.

**What does the trigger do?** Before a student row is inserted or updated, MySQL changes the division to capitals, so "a" is saved as "A".

**How is the percentage calculated?** Present days ÷ (present + absent days) × 100, with one decimal. Below 75% gets a red badge.

**What if the same day is marked twice?** The unique key on (student, date) allows one row, so the second save updates it instead of adding another.

**What happens when MySQL is off?** The server keeps running and the page shows a red "Cannot reach MySQL" message. Nothing is saved.

**Why check the whole class before saving?** So one bad mark (like a future date) saves nothing, instead of saving half the class.

## 9 A 5-day study plan

| Day | Do this |
|---|---|
| Day 1 | Run the app, click everything. Read `schema.sql`, `DbConfig`, `StudentRecord`. Run a few `SELECT`s in MySQL. |
| Day 2 | Read `Database.java` method by method. Find which button uses each one. |
| Day 3 | Read the hooks, then `WebServer.java`. Trace "Save Attendance" (section 5). |
| Day 4 | Read `index.html`: first the HTML, then the CSS, then the JavaScript. Trace "Login" and "Add Student". |
| Day 5 | Do practice tasks 1 to 3. Answer the questions in section 8 out loud, without notes. |

**Stuck?** Add `System.out.println(...)` in Java (it prints in the terminal) or `console.log(...)` in JavaScript (it prints in the browser's developer tools, F12) to see what a variable holds at that moment.